

PRESENTATION PROBLEMS

Instructor: Orhan Özgüner

© Özgüner @CWRU, 2025

Presentation 1

You are designing a pathfinding system for an NPC “mouse” in a simple platforming video game. The game consists of a rectangular board of square blocks, each of which has a height given as an integer. The mouse can begin the game on any square with a height of 0. If it is on a square of height h , it can climb to any adjacent square (horizontally or vertically, but *not diagonally*) with a height $h + 1$. It cannot climb downward or stay on the same level. Your program will need to determine whether, given an arbitrary board B with dimensions $n \times m$ (where $B[i][j]$ is an integer height value), the mouse can ever reach a given height k .

Return **TRUE** if the mouse can travel all the way from height 0 to a square with height k , and **FALSE** if it cannot (note that this is more complicated than just seeing if a square with height k or greater exists on the board).

Hint: This is equivalent to determining if there is a path of *increasing consecutive integers* from 0 to k on the board.

Example:**Input:**

- $m \times n = 3 \times 4$.

- B :

0	1	2	8
2	9	3	4
0	7	6	5

Output: The output for $k \leq 7$ will be **TRUE**, since it is possible to reach as far as height 7 by starting from a 0 cell and always moving horizontally or vertically. However, for any $k > 7$, the result should be **FALSE**, as there is no way to reach the numbers 8 or 9 in this board, when starting from a 0 cell and traversing only consecutive increasing spaces.

Develop an efficient algorithm to decide whether a given height k is reachable on a given grid B .

Problem 1 Solution

Intuition

We will use a modification of DFS to find the path. We iterate through all of the 0 values in the grid, starting a DFS at each one 0 value in the grid. When the node at depth k of the DFS does not equal the previous node +1, we backtrack.

Pseudocode:

Algorithm 1 Pathfinding on a 2D Board

```
1: procedure EXISTS(board, k)
2:    $m \leftarrow \text{length}(\text{board})$  ▷ Vertical size of the board
3:    $n \leftarrow \text{length}(\text{board}[0])$  ▷ Horizontal size of the board
4:   for  $i \leftarrow 0$  to  $m - 1$  do
5:     for  $j \leftarrow 0$  to  $n - 1$  do
6:       if  $\text{board}[i][j] = 0$  then
7:         if SEARCH(board,  $i, j, k, 0$ ) then
8:           return true
9:         end if
10:      end if
11:    end for
12:  end for
13:  return false
14: end procedure

15: function SEARCH(board,  $i, j, k, \text{index}$ )
16:  if  $\text{index} = k$  then
17:    return true
18:  end if
19:  if  $i < 0 \vee i \geq m \vee j < 0 \vee j \geq n \vee \text{board}[i][j] \neq \text{index}$  then
20:    return false
21:  end if
22:   $\text{temp} \leftarrow \text{board}[i][j]$ 
23:   $\text{board}[i][j] \leftarrow \text{'\#'}$  ▷ Mark as visited
24:  for  $(di, dj) \in \{(0, 1), (0, -1), (1, 0), (-1, 0)\}$  do
25:    if SEARCH(board,  $i + di, j + dj, \text{index} + 1, k$ ) then
26:      return true
27:    end if
28:  end for
29:   $\text{board}[i][j] \leftarrow \text{temp}$  ▷ Restore cell
30:  return false
31: end function
```

Note that students can assume the implementation of DFS, but any changes should be stated/provided explicitly in pseudo code.

Runtime:

Runtime of DFS is $\Theta(V + E)$. Although DFS is contained within the for loop, it still only visits each vertex and edge once. Since in the worst case, for each instance of DFS, the algorithm will iterate up to k , we can express $V = \min(k, m \times n)$. However, for each node, there are at most 3 unexplored edges (the 4th is the direction we came from). There are at most $4 \times m \times n$ edges, so the 3^k runtime will dominate for large k (to be exact, when $k > \log(m) + \log(n)$). Thus, $E = \min(3^k, m \times n)$. So, the runtime is $\Theta(\min(k, m \times n) \times \min(3^k, m \times n))$. Since $\min(3^k, m \times n) = \mathbf{O}(\min(3^k, m \times n))$, we can finally simplify this to $\Theta(\min(3^k, m \times n))$.

In the worst case, we will run our modified DFS on all $m \times n$ nodes (or at least every x^{th} node, where x is some arbitrary scalar). Thus, the total runtime is

$$\Theta(m \times n \times \min(3^k, m \times n))$$

Proof of Correctness:

TODO: REVISIT THIS PART

Proof. Let $G = (V, E)$ represent the directed graph where $\mathbb{Z}^+$ is the set of positive integers (including zero) and $E \subseteq \mathbb{Z}^+ \times \mathbb{Z}^+$ represents reachable cells on the board. We need to prove that if there exists a path P of consecutive, increasing integers exceeding k in G , our algorithm will find it.

Lemma 1. *If P exists on the board, its starting square will be initialized as a root node of a DFS tree.*

Proof. Consider the for loops in lines 4 and 5. These loops iterate through each square on the board B which has a value of 0 (which the starting node in P must have), and initializes it as the root node of a DFS tree. The loops will only be terminated if the SEARCH procedure returns true, which means the starting square was initialized as the root node. $\square$

Lemma 2. *The SEARCH procedure correctly determines if the node (i, j) can be the root of the $index \dots k$ subsection of P .*

Proof. We can prove this via induction.

Base Case: If $index = k$ Then the entire sub-path is just the root node. This obviously satisfies our lemma.

Inductive Hypothesis: Assume that the procedure correctly determines if the node (i, j) can be the root of the $index + 1 \dots \ell$ sub-path of P .

Inductive Step: If the indices (i, j) are invalid, then line 18 correctly returns false.

More notably, if the node (i, j) does not equal $index$, then it is not possible that (i, j) is the root of the $index \dots k$ sub-path in P . As such, line 18 correctly returns false.

If $(i, j) = index$, then the function recursively looks at the result for $index + 1$. By the inductive hypothesis, we know this returns the correct result. Thus, via induction and the validity of DFS, we determine the SEARCH procedure correctly determines if the node (i, j) can be the root of the $index \dots k$ sub-path of P . $\square$

From the combination of Lemma 1 and Lemma 2, we can determine that the algorithm correctly finds a path P if it exists anywhere in B

$\square$

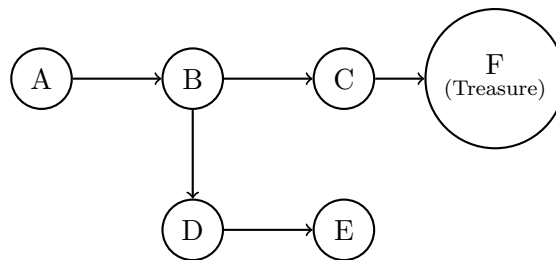
Presentation 2

You are designing a pathfinding system for a “rival” NPC in a simple video game. Both the player and the NPC will be racing each other to find a treasure inside a randomly-generated haunted house, with many different rooms linked by stairs, elevators, hidden doors, etc. The rival should be reasonably lifelike in its behavior, and a fair competitor for the player. It can “remember” rooms it has seen before and store information relating to their status, and should use that information to try to find the treasure as quickly as possible, but it *cannot access a map of the area* or otherwise know information about rooms it hasn’t actually visited.

Example

Input:

- *house* is represented as a graph:



- Starting point: Room **A**.

Output:

- Optimal path to treasure: **A** → **B** → **C** → **F**.

Presentation 2 Solution

Intuition

The ideal behavior for the “rival” is to use a DFS to exhaustively search the house for treasure. It will keep track of the hallways between the rooms, and “mark” those that it has explored. When entering a room in the house, it will mark each exit once, and explore the first unmarked exit, backtracking if it reaches a dead end (whether that is an actual dead end with no exits, or a room where all the exits are marked). It will keep doing this until it finds the treasure.

Pseudocode:

Algorithm 2 Modified DFS-based Haunted House Navigation

```
1: procedure NAVIGATION(house, start, edgeMarks)
2:   for each neighbor n of start do
3:     if edgeMarks[start, n] < 2 then                                ▷ Edge not fully explored
4:       if edgeMarks[start, n] = 1 then
5:         edgeMarks[start, n] ← 2    ▷ We have seen this edge before, from the other side:
6:                                       ▷ Write a second mark on the edge (start, n)
7:       else
8:         edgeMarks[start, n] ← 1    ▷ Write a single mark on the edge (start, n)
9:         NAVIGATION(house, n, edgeMarks)
10:      end if
11:    end if
12:  end for
13:  if treasure is found then
14:    return
15:  end if
16: end procedure
```

Runtime:

The running time of this algorithm is $O(V + E)$, where V is the number of vertices (nodes) in the graph and E is the number of hallways/edges (connections between nodes).

Proof of Correctness:

Proof: We will prove the correctness of the modified DFS-based navigation algorithm by showing that it visits every reachable node/room in the graph. This means that it will eventually find the treasure, if it exists and is reachable from the start.

Claim 1: The algorithm visits every reachable node in the graph.

Proof of Claim 1: We will prove this claim using induction on the number of steps taken by the algorithm.

Base case: Initially, the algorithm starts at the starting room. So, after 0 steps, the starting node is visited.

Inductive hypothesis: Assume that after k steps, the algorithm visits every reachable node up to depth k in the search tree.

Inductive step: We need to show that after $k + 1$ steps, the algorithm visits every reachable node up to depth $k + 1$ in the search tree.

Consider a node u at depth $k + 1$ in the search tree. By the inductive hypothesis, the parent of u at depth k is visited. Since the algorithm visits all the neighbors of a visited node which have 0

or 1 marks, and only marks a neighbor with '2' when it has been visited, the node u will be visited during the next step. Therefore, after $k + 1$ steps, the algorithm visits every reachable node up to depth $k + 1$ in the search tree.

By induction, the algorithm visits every reachable node in the maze.

Claim 2: If the treasure is reachable from the entrance, the algorithm will find it.

Proof of Claim 2: Since the algorithm visits every reachable room in the house (Claim 1), it will eventually visit the treasure room if it is reachable. When the exit is found, the algorithm returns the path to the exit, which consists of the sequence of nodes visited in the search tree from the starting node to the exit node.

Claim 3: If there is no path to the treasure, the algorithm will return when all reachable nodes have been visited.

Proof of Claim 3: If there is no path to the treasure, then the treasure node is not reachable from the starting node. According to Claim 1, the algorithm visits every reachable node in the graph. Since the treasure room node is not reachable, the algorithm will not visit it. In this case, the algorithm will return when all reachable nodes have been visited.

By proving Claims 1, 2, and 3, we have shown that the modified DFS-based navigation algorithm is correct.